DevOps
Advanced Interview Questions & Answers

Q01

Explain the exact sequence of events when a pod gets OOMKilled in Kubernetes — from memory pressure detection to pod restart.

Answer

When a container exceeds its memory limit, the Linux kernel's OOM Killer sends SIGKILL to the process. Kubelet detects the exit code 137 via the container runtime (containerd/CRI-O) and marks the container as OOMKilled. The pod's restartPolicy kicks in — if Always or OnFailure, the container restarts with exponential backoff (10s, 20s, 40s... up to 5 min). The pod itself stays on the same node; it is NOT rescheduled. To debug: check kubectl describe pod for 'OOMKilled' under Last State, then check kubectl top pod and set proper resource.limits.memory. Use VPA (Vertical Pod Autoscaler) to auto-tune limits.

> ***Key Insight:*** *OOMKill restarts the container, not the pod. Node-level memory pressure triggers pod eviction — a different mechanism entirely.*

Q02

Your GitLab CI pipeline runs perfectly in staging but the same Docker image fails in production with a 'permission denied' error. Walk through your complete debugging approach.

Answer

1. Verify the image digest is identical (docker inspect --format='{{.Id}}') — rule out image drift. 2. Compare the runtime context: user (id -u inside container), mounted volumes, securityContext in the pod spec. 3. Check if production has PodSecurityPolicy or OPA/Gatekeeper rules enforcing runAsNonRoot that staging doesn't. 4. Inspect the file permission on the target path — it may be owned by root but the container runs as UID 1001. 5. Check if a read-only filesystem is enforced (readOnlyRootFilesystem: true) in the production deployment spec. 6. Compare environment variables — a wrong SECRET_PATH or TMPDIR can redirect writes to a restricted location. Fix: align securityContext, use initContainers to chown paths, or fix file permissions in the Dockerfile.

> ***Key Insight:*** *Most staging-vs-prod permission bugs come from securityContext differences or PodSecurityAdmission policies, not the image itself.*

Q03

Explain how Kubernetes DNS resolution works when a pod calls another service across namespaces. What are the failure modes?

Answer

Every pod uses CoreDNS (at kube-dns ClusterIP) as its resolver. The search domain list in /etc/resolv.conf is: namespace.svc.cluster.local, svc.cluster.local, cluster.local, then the node's DNS. Cross-namespace: a pod in 'frontend' calling a service in 'backend' must use the FQDN: my-service.backend.svc.cluster.local — the short name 'my-service' will fail because it resolves only within the same namespace. Failure modes: (1) CoreDNS pod down — all DNS fails, check kubectl get pods -n kube-system. (2) NetworkPolicy blocking UDP/53 from the pod to kube-dns. (3) ndots:5 config causing unnecessary external DNS lookups before the cluster domain is tried — add a trailing dot or lower ndots. (4) Service selector mismatch — the service exists but has no Endpoints because labels don't match. Debug with: kubectl exec pod -- nslookup my-service.backend.svc.cluster.local and kubectl get endpoints -n backend.

> ***Key Insight:*** *Always use FQDNs in cross-namespace calls in production. Short names are a silent trap.*

Q04

A terraform apply is causing a state lock that never releases. How do you safely recover without corrupting the state?

Answer

State locks are stored in DynamoDB (for S3 backend) or the backend's locking mechanism. Step 1: Identify the lock — run 'terraform force-unlock' only after confirming NO other apply is running (check CI logs, team comms). Step 2: If using S3+DynamoDB, inspect the lock item in DynamoDB — note the LockID and Operation fields. Step 3: Before any force-unlock, take a manual backup of the state file (aws s3 cp s3://bucket/terraform.tfstate ./backup.tfstate). Step 4: Run 'terraform force-unlock LOCK_ID'. Step 5: Run 'terraform plan' to verify state is consistent before any further apply. Root cause prevention: use short apply windows, set lock_timeout in backend config, and configure CI pipelines to cancel stale jobs rather than abandon them.

> ***Key Insight:*** *Never delete the DynamoDB lock item manually — use force-unlock. Always back up state first.*

Q05

Differentiate between metrics, logs, and traces. Design an observability stack for a microservices system handling 100k req/s.

Answer

Metrics (Prometheus): numeric time-series. Low cardinality, cheap to store. Best for dashboards and alerting. Logs (Loki/ELK): discrete events with context. High cardinality, expensive at scale. Best for debugging. Traces (Jaeger/Tempo): request lifecycle across services. Requires instrumentation. Best for latency analysis. Stack for 100k req/s: Prometheus + Thanos (long-term metrics storage with deduplication across HA replicas). Loki with log sampling (10-20% for INFO, 100% for ERROR). OpenTelemetry collector as the single agent — exports to Jaeger (traces), Prometheus (metrics), Loki (logs). Grafana as unified UI. Alert routing: Alertmanager to PagerDuty (P1), Slack (P2). SLO-based alerting with error budget burn rate rules (burn rate > 14x fires immediately). Use exemplars to link metrics to traces for fast RCA.

> ***Key Insight:*** *At 100k req/s, sampling traces is mandatory. Full trace capture will bankrupt your storage budget.*

Q06

A secret was accidentally committed to a public GitHub repo 3 hours ago. Walk through your complete incident response.

Answer

Assume compromised immediately — 3 hours is an eternity. Step 1: Rotate the secret NOW — revoke the API key/token/password before anything else. Rotation first, investigation second. Step 2: Check access logs for the secret's resource — AWS CloudTrail, GCP Audit Logs, or equivalent — look for unauthorized API calls in the last 3 hours. Step 3: Remove the secret from git history using BFG Repo Cleaner or 'git filter-repo --invert-paths --path file' then force-push. Step 4: Notify security team, document the incident timeline. Step 5: Enable secret scanning (GitHub Advanced Security, truffleHog in pre-commit hooks, GitGuardian). Step 6: Migrate all secrets to a secrets manager (Vault, AWS Secrets Manager) — no secrets in env vars or git, ever. Step 7: Post-mortem with blameless RCA.

> ***Key Insight:*** *git history rewrite does NOT help if the secret was already indexed by bots — rotate first, always.*

Q07

Explain how Horizontal Pod Autoscaler (HPA) works internally — from metric collection to scaling decision to pod creation.

Answer

HPA is a control loop running every 15 seconds (configurable). Flow: 1. HPA controller queries the Metrics API (metrics-server for CPU/memory, or custom metrics adapter for external metrics). 2. It calculates the desired replica count: desiredReplicas = ceil(currentReplicas * currentMetricValue / targetMetricValue). 3. A stabilization window (default 5 min for scale-down, 0 for scale-up) prevents thrashing. 4. HPA patches the Deployment's replica count. 5. Deployment controller creates new ReplicaSets. 6. Scheduler places new pods on nodes. 7. Kubelet starts containers via containerd. Pitfalls: HPA fights VPA if both manage the same resource. CPU-based HPA is reactive — use predictive scaling with KEDA for event-driven workloads. minReplicas should never be 0 for latency-sensitive services — cold start latency will breach SLOs. KEDA extends HPA to scale on Kafka lag, SQS depth, and custom metrics.

> ***Key Insight:*** *HPA + Cluster Autoscaler = full scaling stack. HPA scales pods; CA scales nodes. They must be tuned together.*

Q08

Define error budget, burn rate, and how you would set up multi-window, multi-burn-rate alerting for a 99.9% SLO.

Answer

Error budget = 1 - SLO = 0.1% of requests may fail per month = ~43.8 minutes of downtime. Burn rate = how fast you're consuming the error budget. Burn rate 1 = exactly on pace. Burn rate 14 = consuming a month's budget in ~2 days. Google's multi-window alerting (from SRE Workbook): Alert 1 (Page now): 1h burn rate > 14 AND 5m burn rate > 14. Alert 2 (Page): 6h burn rate > 6 AND 30m burn rate > 6. Alert 3 (Ticket): 3d burn rate > 3 — slow leak, not urgent but needs fixing. Alert 4 (Ticket): 3d burn rate > 1 — budget will be exhausted before month end. Implementation in Prometheus: calculate error ratio per window, compute burn rate as (error_rate / (1 - SLO_target)), then apply multi-window AND logic in alerting rules.

> ***Key Insight:*** *Single-window burn rate alerts have high false-positive rates. Always use the dual-window AND condition.*

Q09

Your microservice experiences intermittent 500ms latency spikes every ~60 seconds. How do you systematically diagnose this?

Answer

60-second periodicity is a strong signal — look for scheduled processes. Approach: 1. Correlate with system metrics: is there a CPU spike, GC pause (for JVM apps), or cron job at that interval? 2. Check Kubernetes liveness/readiness probes — default period is 10s, but some configs use 60s. A slow probe response can cause temporary traffic disruption. 3. Check connection pool exhaustion — many pools have a 60s idle connection timeout. A connection being reaped and re-established causes latency. 4. Check DNS TTL — if service discovery re-resolves every 60s and the upstream changes, requests in-flight fail. 5. Check for leader election events in etcd or ZooKeeper — 60s is a common lease duration. 6. Use distributed tracing to pinpoint which service/span is slow — compare normal vs spike traces. 7. Check Istio sidecar stats — Envoy reports upstream connection reuse metrics that can reveal pool thrashing.

> ***Key Insight:*** *Periodic latency spikes almost always have a timer behind them. Match the period to scheduled operations first.*

Q10

Explain the difference between push-based and pull-based CD. How does ArgoCD ensure GitOps reconciliation without causing deployment storms?

Answer

Push-based CD (Jenkins, GitHub Actions): CI system has write access to the cluster and kubectl-applies on every pipeline run. Simpler but credentials leak risk, no drift detection, hard to audit. Pull-based CD (ArgoCD, Flux): An agent inside the cluster polls Git and applies changes. No external write access, built-in drift detection, full audit trail via git history. ArgoCD reconciliation loop: every 3 minutes (default), the Application Controller compares live cluster state against the desired state in Git. On drift, it can auto-sync or alert. Preventing deployment storms: (1) Sync waves — use sync-wave annotations to order resource creation. (2) Sync windows — restrict auto-sync to business hours only. (3) Resource hooks (PreSync, Sync, PostSync, SyncFail) for database migrations before app rollout. (4) App-of-Apps pattern — one root Application manages child Applications for team-level ownership.

> ***Key Insight:*** *GitOps pull-based CD eliminates the need for cluster credentials in your CI system — a massive security win.*

Q11

Describe 5 Dockerfile best practices that have a direct impact on security and image size in production.

Answer

1. Multi-stage builds: compile in a full image (golang:1.22), copy only the binary to scratch or distroless. Eliminates build tools, compilers, and dev dependencies from the final image. 2. Run as non-root: add USER 1001 before CMD. Prevents container breakout from exploiting root-owned processes. 3. Pin base image digests: FROM ubuntu:22.04@sha256:abc123... not just :22.04. Tags are mutable; digests are not. 4. No secrets in ENV or ARG: secrets passed at build time appear in image history (docker history --no-trunc). Use BuildKit secret mounts (--mount=type=secret) instead. 5. Minimize layers and use .dockerignore: COPY without .dockerignore copies .git, node_modules, and secrets into the image. Each unnecessary layer bloats the image and increases the attack surface. Bonus: scan every image with Trivy or Grype in CI. Fail the build on CRITICAL CVEs.

Key Insight: *A 'FROM scratch' Go binary image can be under 10MB. A default node:18 image is 1.1GB. Size = attack surface.*

Q12

What is Platform Engineering and how does it differ from traditional DevOps? Design an Internal Developer Platform (IDP) for 200 engineers.

Answer

DevOps: culture and practices — developers and ops collaborate, share responsibility. Platform Engineering: building a product (the IDP) that abstracts infrastructure complexity so developers self-serve without needing DevOps expertise. The platform team is the provider; devs are the customers. IDP for 200 engineers: 1. Developer Portal — Backstage as the service catalog. Every service has an owner, SLO, runbook, and dependency map. 2. Golden Paths — opinionated templates for new services: repo + CI pipeline + K8s manifests + monitoring auto-provisioned. 3. Self-service infrastructure — Terraform modules exposed via UI or CLI. Devs request a database or message queue — platform provisions it, injects credentials via Vault. 4. Unified observability — every service auto-enrolled in Prometheus + Grafana + Loki + Tempo with pre-built dashboards. 5. GitOps delivery — ArgoCD ApplicationSets generate an ArgoCD app per service automatically. KPI: reduce time from code-commit to production from days to under 1 hour.

Key Insight: *A good IDP makes the right thing the easy thing. The goal is to eliminate 'platform tickets' entirely.*